

```
In [1]: import pandas as pd
import numpy as np

df = pd.read_csv("messy_sales.csv")
print(df.shape)
```

(300, 6)

```
In [2]: df.head(10)
```

```
Out[2]:
```

	order_id	date	product	price	qty	zip
0	1254	04/06/2026	mug	11.36	4	60614
1	1057	2026-05-30	charger	14.79	3	30303
2	1150	05/06/2026	mug	5.50	1	10001
3	1066	05/30/2026	notebook	NaN	1	98101
4	1114	04/06/2026	webcam	45.59	4	90405
5	1280	04/22/2026	pen set	NaN	4	30303
6	1204	04/18/2026	charger	8.41	2	2134
7	1249	2026-05-15	keyboard	75.04	4	90405
8	1037	2026-05-19	mug	9.31	2	60614
9	1177	2026-05-13	mug	10.11	3	90210

```
In [3]: df.dtypes
```

```
Out[3]: order_id      int64
date          str
product       str
price        float64
qty          int64
zip          int64
dtype: object
```

```
In [4]: df.isna().sum()
```

```
Out[4]: order_id      0
date          0
product       0
price        12
qty          0
zip          0
dtype: int64
```

Observations

1. The price column has 12 NaN values. Until they're filled those rows revenue can't be computed.

2. MM/DD/YYYY and YYYY-MM-DD strings are in the same column and the column is stored as text object instead of a date type.
3. zip is int64, so leading zeros are dropped on zips.

```
In [5]: price_fill_value = df["price"].median()
print("Fill value (median price):", price_fill_value)

df["price"] = df["price"].fillna(price_fill_value)
df.isna().sum()
```

Fill value (median price): 37.53

```
Out[5]: order_id    0
        date        0
        product    0
        price       0
        qty         0
        zip         0
        dtype: int64
```

```
In [6]: n_duplicates = df.duplicated().sum()
print("Duplicate rows found:", n_duplicates)

df = df.drop_duplicates()
print("Shape after dropping duplicates:", df.shape)
```

Duplicate rows found: 8

Shape after dropping duplicates: (292, 6)

```
In [7]: df["zip"] = df["zip"].astype(str).str.zfill(5)
df["zip"].head(10)
```

```
Out[7]: 0    60614
        1    30303
        2    10001
        3    98101
        4    90405
        5    30303
        6    02134
        7    90405
        8    60614
        9    90210
        Name: zip, dtype: str
```

```
In [8]: df[df["zip"].str.startswith("0")].head()
```

Out [8]:

	order_id	date	product	price	qty	zip
6	1204	04/18/2026	charger	8.41	2	02134
13	1108	05/16/2026	desk lamp	21.63	5	02116
21	1090	05/01/2026	keyboard	53.07	1	02116
24	1074	04/30/2026	headphones	46.25	2	02134
26	1147	2026-05-01	headphones	36.37	5	02134

In [9]: `df["date"] = pd.to_datetime(df["date"], format="mixed")`
`df.dtypes`

Out [9]:

order_id	int64
date	datetime64[us]
product	str
price	float64
qty	int64
zip	str
dtype:	object

In [10]: `negative_qty = df[df["qty"] < 0]`
`negative_qty`

Out [10]:

	order_id	date	product	price	qty	zip
202	1140	2026-04-18	webcam	38.69	-5	98101
262	1233	2026-05-09	desk lamp	22.64	-4	10001
297	1025	2026-04-27	keyboard	47.30	-2	02116

In [11]: `df["qty"] = df["qty"].abs()`
`df.loc[[202, 262, 297]]`

Out [11]:

	order_id	date	product	price	qty	zip
202	1140	2026-04-18	webcam	38.69	5	98101
262	1233	2026-05-09	desk lamp	22.64	4	10001
297	1025	2026-04-27	keyboard	47.30	2	02116

In [12]: `df.to_csv("sales_clean.csv", index=False)`
`print("Saved sales_clean.csv with shape", df.shape)`

Saved sales_clean.csv with shape (292, 6)

Cleaning Log

Rows: 300 Missing prices: 12 rows had a missing price. They were filled with the median price (37.53). Price is right-skewed by high-priced items, so the mean would be skewed.

Duplicates: 8 duplicate rows were found and dropped Zip codes: converted zip from int64 to restore leading zeros that were dropped before. Dates: The mixed date formats were parsed into real datetime64 values with `pd.to_datetime(..., format="mixed")`, so the column can be sorted, filtered, and used correctly. Negative quantities: 3 rows had negative qty. You cannot sell a negative number of something. This is sales data, not returns data and there was no return flag. qty and price are normal for these rows otherwise and no other column has a return or cancellation flag, so I treated them as typos and corrected them in place with `.abs()` instead of deleting the rows to avoid discarding real sales.

Data needs to be cleaned properly, because if it misrepresents what's happening and can't be used as an accurate reference it can lead to the wrong decisions being made for important and often high stakes and expensive matters.